

JARVIS — Autonomous AI Operating System

Full-Stack AI/ML Engineering, Multi-Agent Orchestration & Production Architecture

Candidate: Madan Sai

Focus: AI/ML Engineering & Distributed Systems

Status: Production Live on Render & Edge

1. Executive Summary & Vision

JARVIS is a production-grade, 24/7 autonomous AI assistant and cognitive operating system. Unlike fragile LLM wrappers or rigid chatbots, JARVIS combines **cascading multi-model inference**, **in-memory BM25 lexical RAG**, **autonomous task agents**, **cognitive spaced repetition**, and **low-level Linux glibc memory tuning** into a responsive, zero-downtime platform.

Key Engineering Highlights:

- **Resilient Multi-Model Gateway:** Automatic sub-second failover across Groq (Llama 3.3/3.1), Gemini 2.5 Flash, and OpenRouter with circuit breakers.
- **Lexical & Passage RAG Engine:** In-memory BM25 tokenizer and PDF passage extractor achieving sub-10ms query grounding without heavy vector database memory overhead.
- **Autonomous Agent Suite:** 7 specialized modules (Email Triage, Calendar Manager, Job Scout, ATS Resume Alignment, Deploy Watcher, Bill Watcher, Encrypted Diary).
- **Memory Leak Elimination:** Capped glibc malloc arenas (M_ARENA_MAX=2) and trimmed heap transients, flattening container RSS from 512MB+ to a steady ~90MB.

2. 2–5 Minute Interview Walkthrough Script

Use this structured script to present the project with maximum technical clarity:

Phase	Time	Verbatim Presentation Script
Hook & Problem	30s	"Most AI assistant projects are either basic chat wrappers or brittle rule-based bots that break on edge cases and cost a fortune to run. I built JARVIS—a production-grade autonomous operating system that handles my daily workflows: intelligent email triage, calendar scheduling, automated job market scouting with ATS resume alignment, and an interactive cognitive learning engine, running 24/7 with zero-downtime multi-model failover."
Core AI Architecture	90s	"From an AI/ML engineering standpoint, I engineered 4 key systems: 1. Multi-Provider LLM Gateway: A cascading chain using Groq LPU (Llama 3.3/3.1) for ultra-fast generation, falling back to Gemini 2.5 Flash and OpenRouter, protected by advisory circuit breakers and sliding-window rate limiters. 2. Passage RAG Engine: In-memory BM25 retrieval over knowledge docs and PDF notes that prevents hallucinations without vector DB RAM bloat. 3. Autonomous Agents & Safety: 7 specialized agents operating with a strict 'Human-in-the-Loop' hold pattern (pending tables) before dispatching external emails or calendar invites. 4. Cognitive SM-2 Algorithm: Implemented SuperMemo spaced repetition to model the Ebbinghaus forgetting curve for daily technical flight drills."

Systems Hardening & Impact	45s	<i>"On the infrastructure side, I solved real-world memory leaks on Linux containers by capping glibc malloc arenas at 2 (M_ARENA_MAX=2) and running heap trims after PDF parsing, keeping RSS memory under 95MB on free cloud tiers. The UI is a modern React 18 / Vite SPA with continuous voice recognition, and the backend is an async FastAPI service running on Render."</i>
---------------------------------------	-----	---

3. Full Technology Stack & Engineering Justification

Why each specific technology was selected over alternatives:

Component	Technology	Alternative Considered	Why I Chose It (Engineering Rationale)
Backend API	FastAPI (Python 3.11)	Flask, Django	Native asyncio support for high-concurrency external API dispatch (Twilio, Google APIs, LLMs) + automatic Pydantic request schema validation.
Primary Inference	Groq LPU (Llama 3.3/3.1)	OpenAI GPT-4o, Claude 3.5	Sub-second Time-To-First-Token (TTFT) and high token generation speed at near-zero cost, making voice & chat feel instantaneous.
Fallback LLMs	Google Gemini 2.5 Flash & OpenRouter	Local Ollama, Single Provider	Huge 1M+ token context window for dense PDF extraction, combined with multi-provider failover when primary limits hit.
RAG Retrieval	Custom BM25 + pdfplumber	Pinecone, ChromaDB, Weaviate	Zero RAM bloat (under 10MB vs 250MB+ for vector engines) with superior exact keyword/term matching for technical queries.
Database	SQLite + Turso (libSQL)	PostgreSQL, MongoDB	Embedded zero-latency local file development with cloud replication via db_compat without managing heavy database instances.
Scheduler	APScheduler 3.x	Celery + Redis, Cron daemon	In-process cron and interval scheduling with database-backed state restore on server restart—no extra Redis infrastructure needed.
Security & Cryptography	Fernet (AES-128-CBC)	Plaintext DB, bcrypt	Symmetric authenticated encryption for Project Believer, ensuring zero-knowledge confidential diary entries.
Frontend UI	React 18 + Vite + TypeScript	Next.js, Vanilla HTML/JS	Instant Hot-Module Reloading (HMR), strict compile-time type safety across complex dashboard state, and responsive glassmorphism UI.
Voice Interface	Web Speech Recognition API	Whisper API (Cloud)	Zero-latency client-side continuous voice capture with custom closing phrase detection ('it's over') and silence fallbacks.

4. Specialized Autonomous Agent Fleet

Detailed overview of the modular agent fleet deployed in the system:

Agent Module	Key Architectural Capabilities & Workflows	Safety & Persistence
Email Triage Agent email_triage.py	Watches Gmail inbox via OAuth2, analyzes priority using LLM, extracts actionable tasks, learns user tone from prior edits, and generates response drafts.	Hold Table: pending_drafts (Requires explicit approval before sending).
Calendar Agent calendar_agent.py	Checks schedule availability, resolves conflicting times, and formats event descriptions with attendees.	Safety Hold: pending_calendar_events holds external invites until approved.
Job Scout Agent job_scout_agent.py	Daily cron scraping across Adzuna, Remotive, and JSearch; deduplicates against seen_jobs; LLM pre-filters and ranks high-match roles.	Tracking: Auto-populates applications table with Kanban statuses.
Resume ATS Agent resume_ats_agent.py	Extracts requirements from job descriptions, scores master resume, reframes bullets to STAR/XYZ format, and generates downloadable tailored resumes.	Conservative: Never fabricates skills; outputs honest gap reports.
Deploy Watcher deploy_watcher_agent.py	Monitors GitHub commits, tracks container RSS memory, probes health endpoints, and alerts on anomalies or memory leaks.	Telemetry: Logs memory trends and HTTP status codes in SQLite.

Bill Watcher bill_watcher.py	Natural language bill registration ('rent 25000 on the 1st'), recurring cycle tracking, notification deduplication, and paid cycle rolling.	Web Inbox: Generates in-app notifications and dashboard reminders.
Cognitive Learning learning_engine.py	Generates 10-minute Socratic flight drills with broken code, evaluates user solutions with diagnostic scoring, and calculates SM-2 mastery schedules.	SM-2 Radar: Tracks ease factor and interval days per concept.

5. Production Engineering & Low-Level Performance Tuning

One of the strongest technical stories of this project is solving the **Linux glibc malloc arena ratcheting bug** that caused container Out-Of-Memory (OOM) crashes on Render.

The glibc Memory Arena Problem & Fix:

Problem: In Linux glibc, multi-threaded background workers (e.g. `requests.get`, PDF parsers) create separate memory arenas (up to 8× CPU cores). When large buffers (PDF bytes, scraped HTML) are freed in Python, glibc hoards the memory arenas instead of returning them to the OS. Over time, RSS ratchets past 512MB.

Engineering Solution:

1. Capped malloc arenas at startup via `ctypes.CDLL('libc.so.6').mallopt(-8, 2)` (equivalent to `M_ARENA_MAX=2`).
2. Added explicit `ctypes.CDLL('libc.so.6').malloc_trim(0)` calls immediately after transient PDF parsing and digest runs.

Result: Server idle RSS memory dropped and flattened to a stable **~64–90 MB** permanently.

6. Key Interview Questions & Winning Answers

Q1: How do you prevent hallucinations in your LLM agent workflows?

Answer: I implement a three-layer defense: First, **Grounding via BM25 RAG**—I retrieve verified top-k passages from internal notes and inject them as system context. Second, **Strict JSON Schema Enforcement**—prompts require exact JSON keys, and outputs are stripped of markdown fences before parsing. Third, **Conservative Tailoring Constraints**—for example, in the Resume ATS agent, the system prompt explicitly forbids hallucinating tools not in the master resume, producing an honest gap report instead.

Q2: Why did you build an in-memory BM25 engine instead of using a Vector Database (like Pinecone/Chroma)?

Answer: Resource efficiency and query precision. Heavy vector DBs and embedding models consume 250MB+ RAM and add external latency/cost. In our personal agent architecture, documents are dense technical notes where exact keyword terms (e.g. function names, specific dates, error codes) matter. BM25 runs in sub-10ms with under 5MB RAM and zero external dependencies.

Q3: How do you handle natural language intent routing without brittle regex matching?

Answer: Early hardcoded keyword matching broke on natural conversational phrasing. I replaced it with an AI Intent Classifier (`MEMORY_INTENT_PROMPT`). Incoming text is classified into structured intent enums (`JOB_SEARCH`, `CALENDAR_ACTION`, `BILL_ACTION`) by a high-speed Groq model. If confidence is high, it routes to the dedicated agent; otherwise, it falls back to conversational chat.

Q4: What is the SuperMemo SM-2 algorithm and how did you implement it in the Learning Engine?

Answer: SM-2 models the Ebbinghaus forgetting curve by calculating optimal review intervals. When a user completes a Socratic flight drill, their score (0-100) translates to a quality score (0-5). We update the concept's *Ease Factor* (minimum 1.3) and scale the *Interval Days* (1 day → 6 days → interval × ease factor). This guarantees high-difficulty concepts are reviewed sooner, while mastered topics are spaced further out.